

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Направление: 02.03.02 «Фундаментальная информатика и информационные технологии»

ООП: Большие данные и распределенная цифровая платформа

ОТЧЕТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

Тема: Распределенный инференс глубоких нейросетевых моделей

Выполнил:

Учебная группа

Колосков Виктор Юрьевич

22Б.15-пу

Научный руководитель:

Должность:

Першин Антон Юрьевич

доцент кафедры фундаментальной

информатики и информационных систем

Ph. D.

Ученая степень:

Председатель комиссии:

Должность:

Утешев Алексей Юрьевич

профессор, заведующий кафедрой

фундаментальной информатики и

распределенных систем

Доктор физико-математических наук

Ученая степень:

Санкт-Петербург 2025

СОДЕРЖАНИЕ

Введение	3
Постановка задачи	4
1. Способы параллелизма моделей	5
1.1. Тензорный параллелизм	5
1.2. Конвейерный параллелизм	6
1.3. Экспертный параллелизм	8
2. Методы оптимизации распределенного инференса	9
2.1. Выгрузка параметров на CPU и NVMe	9
2.2. Оптимизация коллективных операций	10
2.3. Оптимизации путей активации экспертов для MoE архитектур	10
3. Инструменты для реализации распределенного инференса	13
3.1. MindSpore	13
3.2. DeepSpeed	13
3.3. Megatron-LM	14
4. Практические эксперименты	15
4.1. Тензорный параллелизм и параллелизм по данным для плотных моделей	15
4.2. Использование ZeRO с CPU и NVMe	16
4.3. Использование тензорного параллелизма с ZeRO и без него	17
4.4. Использование параллелизма по данным и тензорного параллелизма с ZeRO	18
4.5. Использование тензорного параллелизма вместе с экспертным параллелизмом	19
Вывод	20
Список использованной литературы	21
Приложение А	23
Приложение В	23

Введение

В современную эпоху активного развития искусственного интеллекта и глубокого машинного обучения большие нейросетевые модели становятся неотъемлемой частью решения множества сложных задач, таких как распознавание образов, обработка естественного языка и автономное управление.

Инференс нейросетевых моделей — это процесс использования уже обученной нейронной сети для предсказания результатов на новых, ранее невидимых данных. В отличие от этапа обучения, когда модель настраивает свои веса на основе обучающих данных, во время инференса модель остаётся фиксированной, и её задача — эффективно применять накопленные знания для решения поставленной задачи.

Однако увеличение сложности и размеров моделей часто приводит к проблемам, связанным с чрезмерными временными затратами на инференс или невозможностью размещения модели в памяти графического ускорителя (GPU). Одним из решений этой проблемы является распределение нейросетевой модели между несколькими GPU. Такой подход называется параллелизмом на уровне модели.

Вопросы эффективного распределения нейросетевых моделей по вычислительным устройствам становятся особенно актуальными в условиях роста требований к вычислительным ресурсам и необходимости выполнения вычислений в реальном времени. Распределённый инференс позволяет использовать несколько устройств (GPU, CPU) для преодоления ограничений по памяти и вычислительной мощности, а также для сокращения времени отклика системы.

При распределённом инференсе возникает необходимость разделить матрицы весов модели между различными устройствами. Для этого требуется определённым образом «разрезать» нейронную сеть и распределить эти части по доступным ресурсам. Существует два способа разрезания модели: горизонтальный (межслойный) и вертикальный (внутрислойный).

Горизонтальный способ достаточно прост в реализации, однако может быть не очень эффективным. Дело в том, что при таком подходе обработка данных на каждом слое происходит только на одной видеокарте, вследствие чего другие устройства могут простаивать, что приводит к неэффективному использованию вычислительных ресурсов. Вертикальный способ, напротив, позволяет задействовать одновременно несколько видеокарт, что значительно повышает производительность вычислений. Однако его реализация является более сложной.

В данной работе рассматриваются различные методы распределённого инференса глубоких нейросетевых моделей, а также инструменты, реализующие эти подходы.

Постановка задачи

Цель: Исследовать различные подходы к распределенному инференсу и оценить эффективность соответствующих инструментальных решений.

Задачи:

1. Исследование тезного параллелизма.
2. Изучение конвейерного параллелизма.
3. Анализ параллелизма экспертов для архитектур МоЕ.
4. Изучение процесса инференса моделей на гетерогенных вычислительных системах.
5. Обзор и анализ инструментов, реализующих разные способы параллелизации моделей.

1. Способы параллелизма моделей

1.1. Тензорный параллелизм

Тензорный параллелизм — это метод параллельного разбиения модели, при котором тензор параметров каждого слоя распределяется между несколькими процессорами.

Впервые тензорный параллелизм был представлен в статье о модели Megatron-LM [6] как способ обучения больших языковых моделей на значительном количестве видеокарт. В данной работе для обучения моделей BERT и GPT-2 использовалось 512 видеокарт. Модель основана на архитектуре трансформера, за которой следует двухслойный перцептрон. На рисунке 1 показан способ разделения перцептрона.

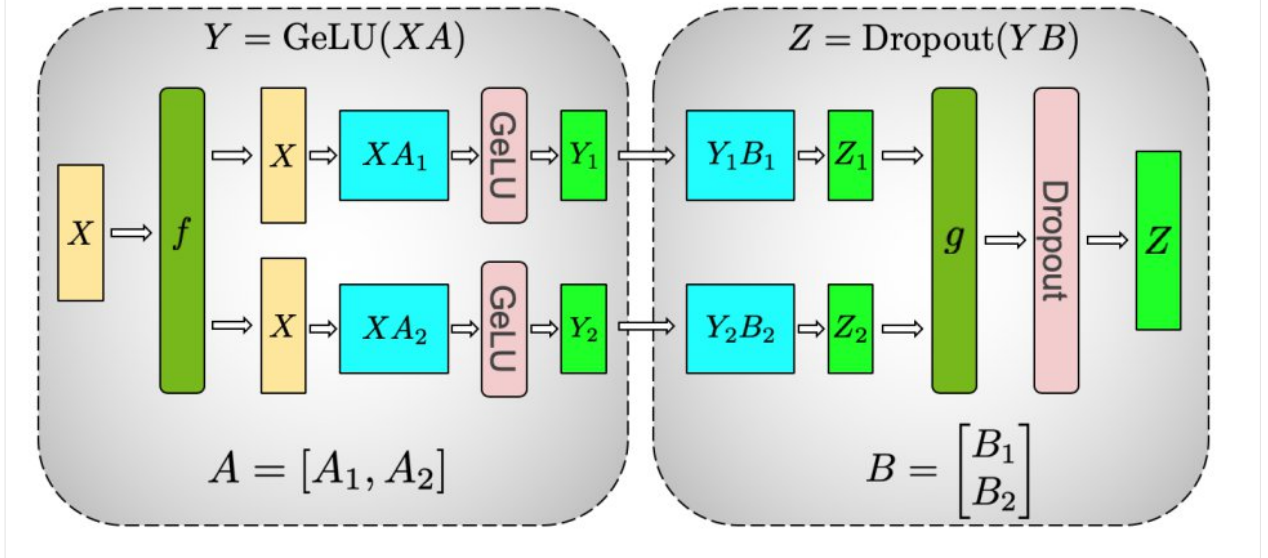


Рисунок 1. Способ разделения перцептрона [6]

Y может быть представлен следующим образом:

$$Y = \text{GeLU}(XA) \quad (1)$$

Существует возможность разделить матрицу весов по столбцам: $A = [A_1, A_2]$. Тогда Y представима в следующем виде:

$$Y = [\text{GeLU}(XA_1), \text{GeLU}(XA_2)] \quad (2)$$

Так как следующую операцию умножения можно также провести в разделенном для Y виде (1), нет необходимости на этом этапе добавлять какие-либо точки синхронизации. Следующая матрица весов разделяется по строкам. Таким образом матрицу Z можно представить в виде:

$$Z = \text{Dropout}(Z_1 + Z_2) = \text{Dropout}(Y_1 B_1 + Y_2 B_2) \quad (3)$$

Следовательно, для получения матрицы Z необходимо добавить точку синхронизации и применить оператор $all - reduce(g)$, задача которого состоит в том, чтобы собрать полученные матрицы со всех GPU и произвести операцию редукции.

На рисунке 2 представлен способ разделения блока self-attention.

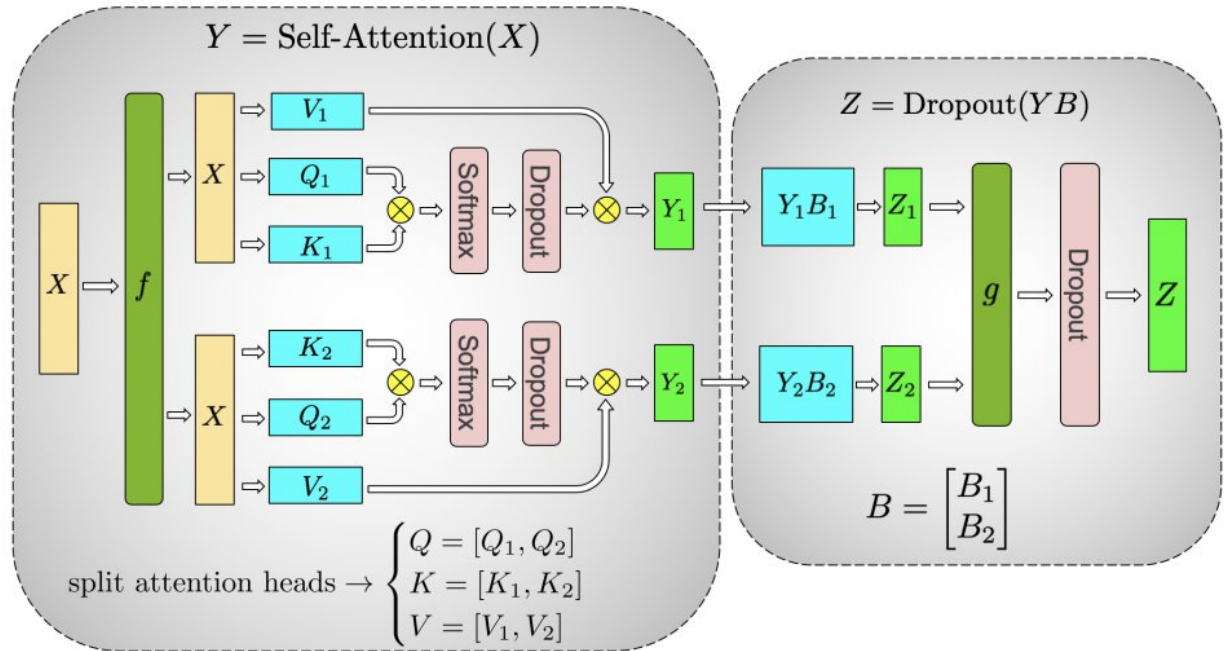


Рисунок 2. Тензорный параллелизм self-attention [11]

В этом случае разделяются матрицы Q, K, V по столбцам и вычисления производятся независимо на каждом GPU. После этого получаем матрицу $Y = [Y_1, Y_2]$. Второй этап аналогичен разделению перцептрона.

Таким образом, данный подход позволяет существенно повысить скорость обучения и инференса модели. Однако его эффективность зависит от разработки надёжного механизма разделения матриц весов.

Вместе с тем, данный подход обладает существенной проблемой масштабируемости: эффективное использование данного вида параллелизма ограничено пределами одного узла видеокарт. Это связано с необходимостью синхронизации и межузловой передачи данных на определённых этапах вычислений, что приводит к значительному снижению скорости обработки информации при распределении вычислений между несколькими узлами.

1.2. Конвейерный параллелизм

Конвейерный параллелизм представляет собой подход к распараллеливанию модели, при котором каждый слой или группа последовательных слоев размещается на отдельном вычислительном узле. Последовательные пакеты данных подаются в конвейер таким образом, чтобы обеспечить его постоянную загрузку. Обмен данными между узлами ограничивается передачей активаций слоёв, где происходит разделение модели, а также соответствующих градиентов этих активаций. [5]

На рисунке 3 представлен пример конвейерного параллелизма.

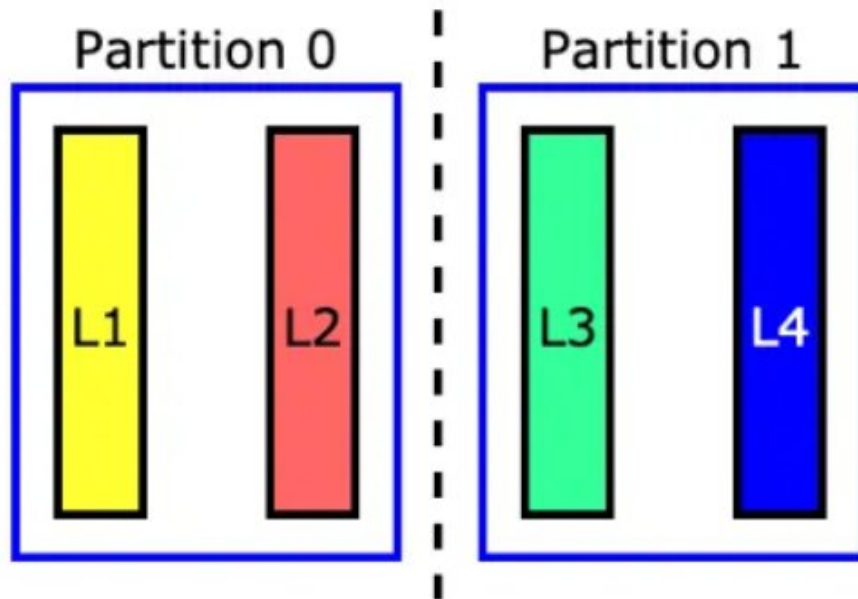


Рисунок 3. Конвейерный параллелизм

Этот тип разделения модели обеспечивает высокую эффективность при её масштабировании между вычислительными узлами, поскольку операции на каждом слое не требуют выполнения дополнительных коллективных коммуникаций. Вместе с тем, конвейерный параллелизм имеет существенный недостаток — значительная часть графических процессоров простаивает в момент выполнения вычислений на отдельном слое. Более того, размер отдельного слоя модели может превышать объём видеопамяти одного устройства, что ограничивает применимость подхода.

Для устранения первой проблемы входной батч разбивается на несколько микробатчей, которые последовательно подаются в модель, позволяя начать обработку следующего микробатча до завершения вычислений предыдущего по всей глубине сети. Это позволяет более эффективно загружать вычислительные ресурсы и сокращает время простоя устройств. Для решения второй проблемы, связанной с ограниченной памятью отдельного GPU, внутри одного узла применяется тензорный параллелизм, обеспечивающий распределение отдельных слоёв между несколькими видеокартами. Между узлами, в свою очередь, сохраняется использование конвейерного параллелизма, что в совокупности позволяет эффективно распределять модель как внутри узла, так и между узлами вычислительного кластера. Иллюстрация данного подхода представлена на рисунке 4.

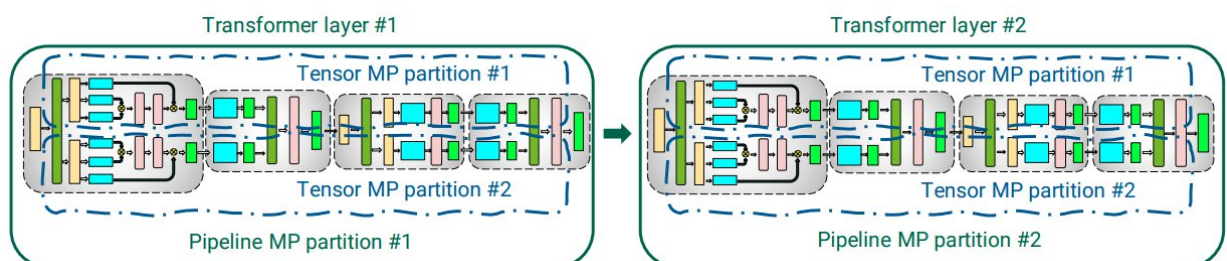


Рисунок 4. Использование тензорного параллелизма вместе с конвейерным. [11]

1.3. Экспертный параллелизм

Для начала рассмотрим архитектуру моделей смеси экспертов (MoE).

Mixture of Experts (MoE) — это архитектура нейронных сетей, которая позволяет эффективно масштабировать модель, не увеличивая вычислительные затраты пропорционально размеру. Основная идея MoE заключается в том, чтобы разделить вычисления между несколькими «экспертами», каждый из которых специализируется на определённом типе входных данных. При обработке запроса активируются только те эксперты, которые наиболее релевантны текущей задаче. Такой подход делает модель более производительной и масштабируемой, что особенно важно в современных больших языковых моделях. На рисунке 5 представлен пример MoE архитектуры для трансформерного слоя.

В контексте моделей трансформеров MoE состоит из двух основных элементов [14]:

1. Вместо слоев плотной сети прямой связи (FFN) используются разреженные слои MoE. В слоях MoE есть определенное количество «экспертов», где каждый эксперт является нейронной сетью. На практике экспертами являются FFN, но они также могут быть более сложными сетями или даже самим MoE.
2. Network gate или router, который определяет, какие токены отправляются какому эксперту. Например, на изображении ниже токен "More" отправляется второму эксперту, а токен "Parameters" отправляется в первую сеть. Как мы узнаем позже, мы можем отправить токен более чем одному эксперту. Как направить токен эксперту - одно из важных решений при работе с MoE - маршрутизатор состоит из изученных параметров и предварительно обучен одновременно с остальной сетью.

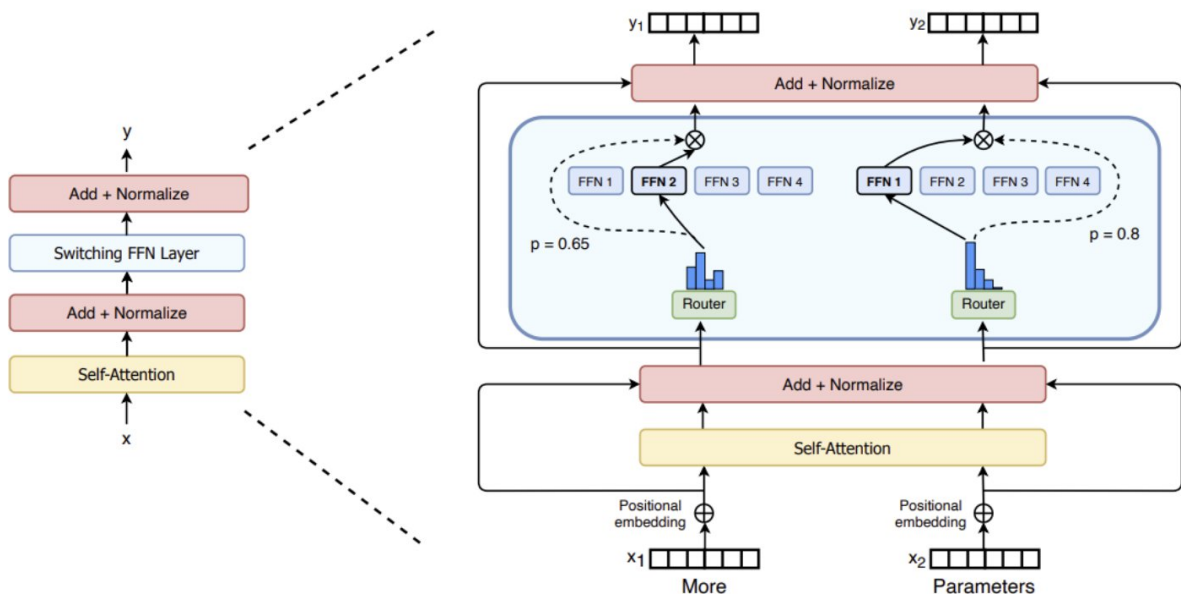


Рисунок 5. Пример архитектуры MoE для трансформерных моделей.[14]

Идея MoE параллелизма заключается в разделении экспертов по группам, каждая из которых будет находиться на одном GPU или узле. Это позволит значительно снизить объем используемой памяти на отдельном GPU и значительно повысить скорость

инференса за счет того, что эксперты в пределах одного слоя могут выполнять вычисления независимо.

Одной из первой реализацией этого параллелизма была представлена в рамках фреймворка GShard [15]. В данном подходе экспертные слои разделяются на логические группы, каждая из которых распределяется по отдельным вычислительным устройствам (например, GPU). При этом неэкспертные слои модели дублируются на всех устройствах, что позволяет использовать также и параллелизм по данным.

Однако, этот способ имеет существенную проблему. На рисунке 6 представлена архитектура сети с несколькими MoE слоями. Из нее видно, что при переходе между слоями необходимо произвести операцию *AlltoAll*. Эта операция требует полной синхронизации и обмена данными между участниками, вследствие чего является одной из основных проблем параллелизма такого вида.

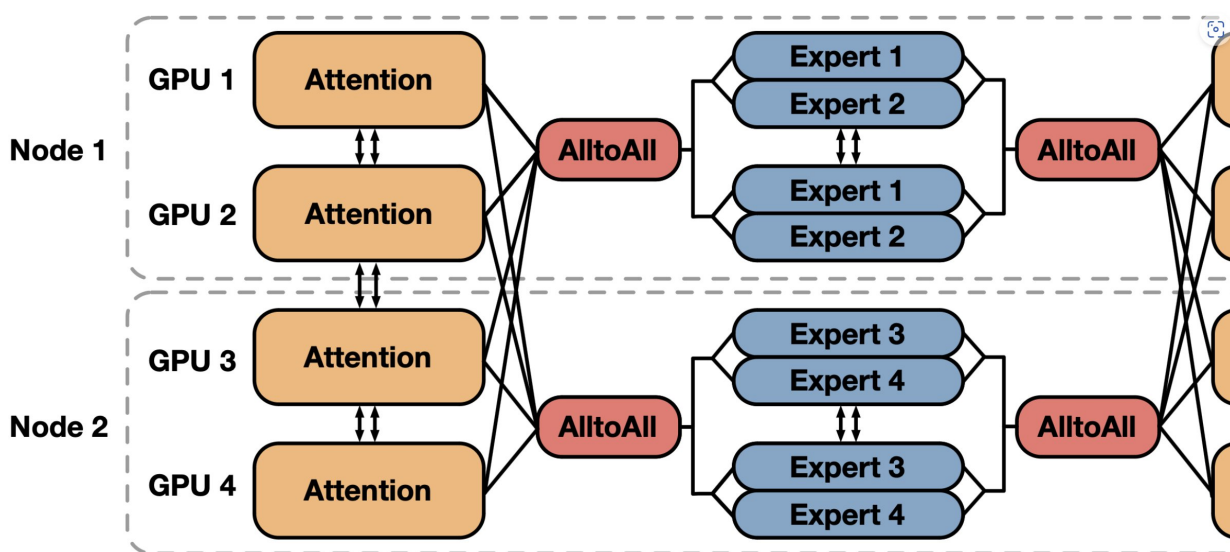


Рисунок 6. Экспертный параллелизм [1]

2. Методы оптимизации распределенного инференса

2.1. Выгрузка параметров на CPU и NVMe

Несмотря на то, что существуют различные способы разделения моделей, многим исследователям их использование все еще недоступно. Это связано с тем, что даже не очень большие модели могут требовать десятки или даже сотни GPU для использования. В реальных условиях количество доступных вычислительных устройств часто строго ограничено, что делает традиционные методы распределённого вывода неприменимыми.

В таких ситуациях одним из возможных решений становится использование более медленной, но доступной по объёму оперативной или дисковой. Основная идея заключается в том, чтобы динамически загружать на GPU только те части модели, которые необходимы для текущих вычислений, а затем выгружать их обратно в память хоста после завершения обработки. Такой подход позволяет значительно снизить

требования к видеопамяти и выполнять инференс моделей, размер которых превышает объём локальной памяти GPU [12].

Кроме выгрузки параметров модели, в память хоста или на диск также может быть перемещена часть промежуточных вычислений, что позволяет дополнительно снизить нагрузку на видеопамять. В частности, при инференсе трансформерных моделей широкое применение находит кеширование ключей и значений (KV-cache) — метод, позволяющий сохранять результаты некоторых промежуточных вычислений для каждого токена [4]. Благодаря этому при добавлении нового токена к уже существующей последовательности нет необходимости повторно обрабатывать всю входную последовательность целиком, достаточно произвести вычисления только для нового токена. Однако при работе с длинными последовательностями объём данных, хранимых в KV-кэше, может быть значительным, что создаёт дополнительную нагрузку на GPU-память. Для решения этой проблемы применяется выгрузка содержимого KV-кэша в оперативную или дисковую память по мере необходимости.

2.2. Оптимизация коллективных операций

При выполнении инференса происходит значительное количество синхронизаций и межпроцессорного обмена данными между GPU. Вместе с тем, отдельные коллективные операции могут быть недостаточно хорошо оптимизированы для использования в составе нейросетевых моделей. С учётом специфики задачи инференса появляется возможность самостоятельной оптимизации указанных коллективных операций, что позволяет снизить их вычислительную сложность и ускорить их выполнение. Так, например, DeepSpeed [11] пытается оптимизировать операцию AlltoAll путем использования иерархической структуры. На рисунке 8 проиллюстрирован пример его работы. Сначала происходит преобразование макета данных, затем внутриузловой AlltoAll. После еще одного преобразования шаблона данных, реализуется внешний AlltoAll. Это сокращает количество связей с $O(p)$ до $O(g + \frac{p}{g})$, где p и g — общее количество GPU и число GPU на одном узле.

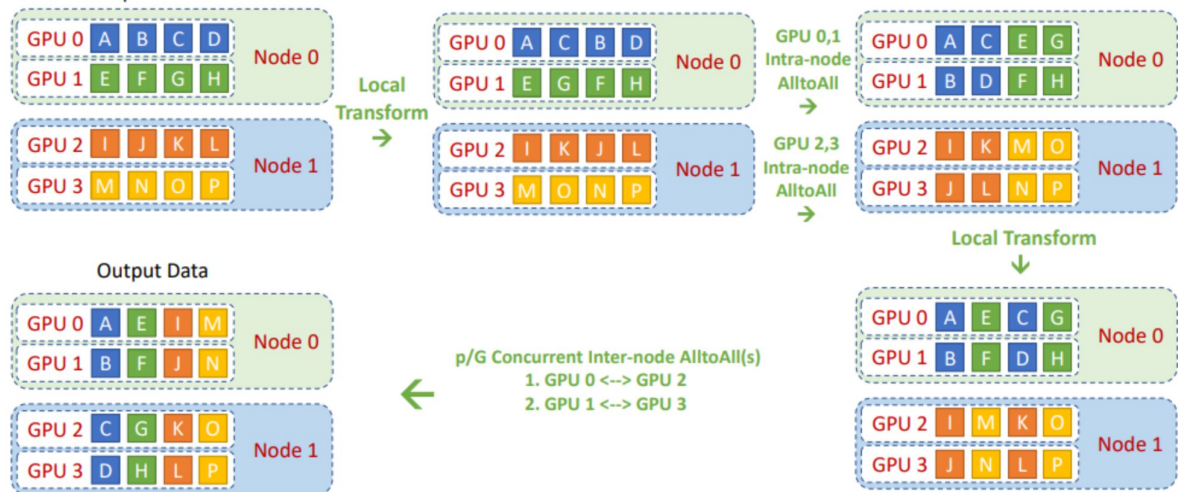


Рисунок 7. Иерархический MoE.

2.3. Оптимизации путей активации экспертов для MoE архитектур.

В MoE архитектурах операции AlltoAll происходят из-за того, что необходимо передать эксперту соответствующие токены для вычислений. Но, если токен уже находится

на GPU, то нет необходимости в его передаче. Следовательно, можно попытаться предсказывать на основе семантики токена к какому эксперту он отправиться. К тому же, между выходом предыдущего экспертного слоя и следующим за ним может быть аналогичная зависимость. Следовательно, если оптимально «пересадить» экспертов между GPU, то существует возможность уменьшить количество данных для передачи. В статье [17] приводятся такие способы оптимизации коллективных операций.

Для предсказания типа «токе-эксперт» используется простая модель, построенная на условных вероятностях:

$$P(E_k^L | x_j) = \frac{T_{j,k}^L}{\sum_{k=1}^{N^L} T_{j,k}^L} \quad (4)$$

Где $T_{j,k}^L$ – количество раз, когда токен x_j был маршрутизирован эксперту E_k^L .

Используется конечное количество токенов, записанных в словаре, которые были получены при более раннем инференсе. В случае появления нового токена, при помощи метрики косинусного расстояния вычисляется наиболее близкий к нему токен, находящийся в словаре.

Вторая оптимизация состоит в объединении токенов в кластеры на основе вероятностей, полученных на предыдущем шаге. Эту проблему можно описать в виде задачи линейного программирования:

$$L = \theta \sum_{i=1}^E \left| \sum_{j=1}^t (R_{ij} a_j) - \frac{S}{E} \right| + (1 - \theta) \sum_{i_1 \neq i_2} \left(\sum_{j=1}^t \sum_{k=1}^N (R_{i_1 j} C_{i_2 k} P_{jk} a_j) \right) \rightarrow \min \quad (5)$$

$$\sum_{i=1}^R R_{ij} = 1, j = 1..t$$

$$\sum_{i=1}^E P_{ij} = 1, j = 1..N$$

$$\sum_{j=1}^N C_{ij} = \frac{N}{E}, i = 1..t$$

Где t – количество уникальных токенов, N – количество экспертов на слой. E – количество кластеров, a_j – частота токена j . P_{jk} – прогнозируемая вероятность того, что токен j активирует эксперта k , $R_{ij} \in \{0,1\}$ – размещение токена j в кластере i . $C_{ij} \in \{0,1\}$ – размещение эксперта j в кластере i .

На рисунке 9 представлена карта частоты активаций эксперта в зависимости от токена на первом слое DeepSeek-MoE

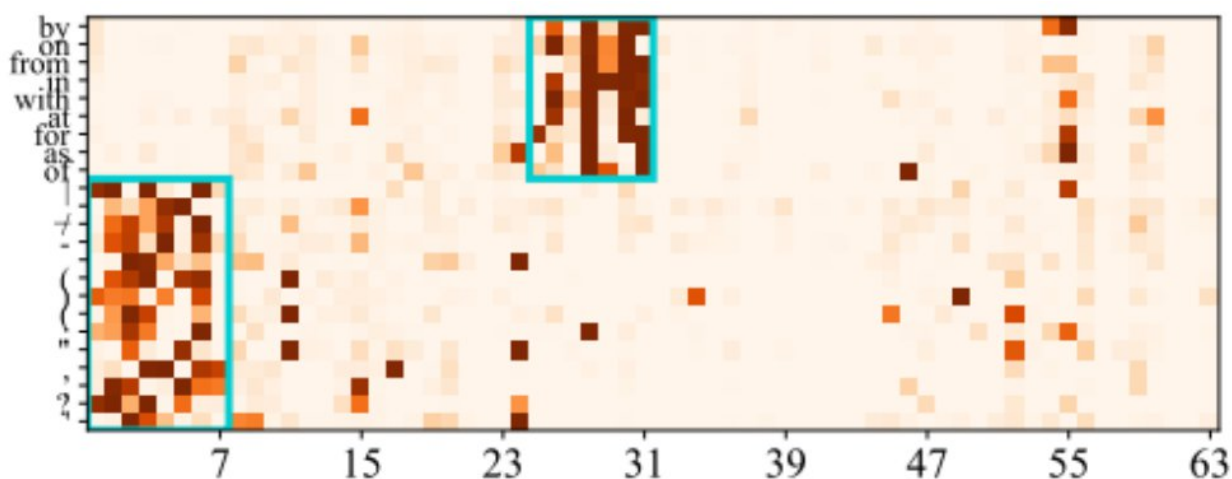


Рисунок 8. Частота активации эксперта от токена на первом слое модели DeepSeek-MoE

Для моделирования связи между кластерами используется также модель на условных вероятностях:

$$P(D_k^L | D^{L-1}, \dots, D^{L-n}) \quad (6)$$

Где D^l – индекс устройства, где выбранному эксперту соответствует l -й слой произвольного токена.

На рисунке 10 представлена карта частоты активаций эксперта на слое 5 в зависимости от активации эксперта на слое 4 Mixtral-8x7b.

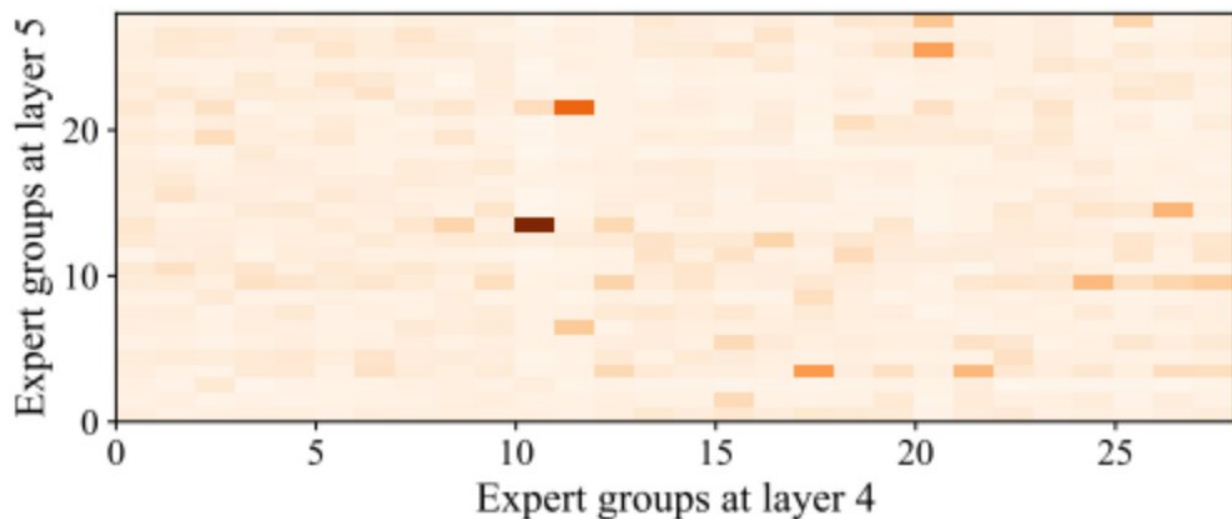


Рисунок 9. частота активаций эксперта на слое 5 в зависимости от активации эксперта на слое 4 Mixtral-8x7b. частоты активаций эксперта на слое 5 в зависимости от активации эксперта на слое 4 Mixtral-8x7b.

3. Инструменты для реализации распределенного инференса

В рамках работы рассмотрены три фреймворка, предоставляющих инструменты для распределенного инференса: MindSpore [1], DeepSpeed [7], Megatron-LM [8].

3.1. MindSpore

MindSpore — это открытая и гибкая вычислительная фреймворк-платформа для разработки и выполнения моделей искусственного интеллекта, созданная компанией Huawei . Платформа распространяется с полностью открытым исходным кодом и, помимо GPU способна работать на NPU процессорах Ascend.

Одной из ключевых особенностей MindSpore является её способность выполнять распределённое обучение без использования сторонних библиотек, таких как PyTorch , для реализации коллективных операций. Вместо этого платформа использует библиотеку OpenMPI. Однако, это также является и его проблемой, поскольку нет возможности применять оптимизации других фреймворков, созданных под torch, а также некоторые инструменты, связанные с предоставлением обученных моделей могут быть недоступны.

MindSpore обладает встроенной поддержкой продвинутых методов распараллеливания, включая тензорный конвейерный параллелизм и экспертный параллелизм , а также позволяет использовать их комбинации для эффективного масштабирования вычислений при обучении крупных моделей. Платформа также предоставляет унифицированное промежуточное представление модели — MindIR , которое выступает в роли абстрактной прослойки между Python- и C++-компонентами фреймворка, обеспечивая гибкость и переносимость моделей.

Кроме того, MindSpore поддерживает JIT компиляцию, позволяющую динамически оптимизировать выполнение функций, что способствует значительному ускорению вычислений и повышению общей производительности.

Несмотря на широкие функциональные возможности и высокую степень интеграции с аппаратными средствами, развитие фреймворка в настоящее время сосредоточено в первую очередь на поддержке собственных чипов Huawei серии Ascend. Поддержка устройств других производителей, в частности GPU, остаётся менее приоритетной: используются устаревшие версии технологий, что ограничивает возможность применения современных оптимизаций и библиотек NVIDIA.

3.2. DeepSpeed

DeepSpeed — это масштабируемая библиотека с открытым исходным кодом, разработанная Microsoft Research для ускорения и оптимизации процесса обучения глубоких нейронных сетей. Она предоставляет мощные инструменты для эффективного использования вычислительных ресурсов, что особенно важно при работе с крупномасштабными моделями.

Реализована при помощи torch, благодаря чему способно использовать множество оптимизаций. К тому же хорошо интегрируется с другими фреймворками: HuggingFace Transformers, Fairseq, Megatron-LM, Lightnig AI.

Deespeed поддерживает конвейерный и экспертный параллелизм. Также благодаря технологии ZeRO-inference [9] способен выгружать части модели на CPU и диск. При этом используется библиотека для асинхронной передачи данных между RAM и дисковой памятью DeepNVMe, что дает возможность перекрывать коммуникационные операции вычислительными и существенно повысить скорость инференса.

Еще одним важным преимуществом DeepSpeed является поддержка тензорного параллелизма для моделей из библиотеки Hugging Face Transformers с возможностью его применения «на лету», без предварительной модификации архитектуры. Это позволяет эффективно распределять вычисления между GPU, обеспечивая масштабируемость и высокую производительность.

Однако такой подход связан с увеличением потребления памяти из-за дублирования данных и межпроцессного взаимодействия. В других фреймворках использование тензорного параллелизма, как правило, требует предварительного разбиения модели вручную, что усложняет интеграцию и снижает гибкость.

Однако стоит отметить, что данный инструмент еще достаточно сырой и возникает множество проблем с его использованием. К примеру, нет возможности использования тензорного параллелизма вместе с ZeRO, из-за чего приходится использовать дополнительные фреймворки по типу Megatron-LM.

Следует отметить, что несмотря на широкие возможности DeepSpeed, инструмент всё ещё находится на этапе активной разработки и в ряде аспектов является недостаточно зрелым. Это проявляется в виде различных ограничений и сложностей при практическом использовании. Например, на текущий момент отсутствует полноценная поддержка одновременного использования тензорного параллелизма и технологии ZeRO, что сужает возможности для эффективного масштабирования моделей на больших кластерах.

В связи с этим, для реализации более сложных сценариев распределённого обучения, таких как гибридный параллелизм с высокой степенью оптимизации, зачастую приходится прибегать к дополнительным фреймворкам, таким как Megatron-LM, которые обеспечивают необходимый уровень гибкости и производительности.

3.3. Megatron-LM

Megatron-LM — это исследовательский проект, разработанный компанией NVIDIA, предназначенный для эффективного обучения крупномасштабных языковых моделей на основе архитектуры Transformer. Фреймворк изначально ориентирован на решение задачи масштабирования моделей по числу параметров и вычислительных ресурсов, что делает его особенно актуальным в условиях роста сложности современных моделей глубокого обучения.

Megatron-LM поддерживает два вида параллелизма: конвейерный и тензорный. К тому же имеет тесную интеграцию с оборудованием NVIDIA, благодаря чему может использовать множество оптимизаций. Может хорошо интегрироваться с

DeepSpeed, благодаря чему существует возможность обучать такие модели как Megatron-Turing NLG.

Однако, имеет две важные проблемы. Во-первых, он требует значительного уровня экспертизы в области распределённого обучения и программирования на уровне GPU. Во-вторых, этот инструмент изначально разрабатывался для обучения моделей, вследствие чего многие инструменты для ускорения инференса недоступны и их необходимо реализовывать самостоятельно.

4. Практические эксперименты.

Эксперименты проводились на вычислительном кластере, оснащённом двумя GPU NVIDIA GeForce RTX 2080 Ti, каждый из которых обладал 11 ГБ видеопамяти, а объём оперативной памяти системы составлял 32 ГБ. Для исследования влияния архитектурных параметров на масштабируемость моделей использовались две конфигурации: в случае плотной модели изменялось количество слоёв, а в случае архитектуры МоЕ дополнительно варьировалось число экспертов. Подробные спецификации исследованных моделей, а также формулы для расчёта их размера, приведены в Приложении А. В приложении В представлены результаты всех

4.1. Тензорный параллелизм и параллелизм по данным для плотных моделей

Рассмотрим сравнение тензорного параллелизма и параллелизма по данным. На рисунке 10 приведено сравнение объёма видеопамяти, потребляемой каждым GPU, при использовании указанных стратегий распределения вычислений.

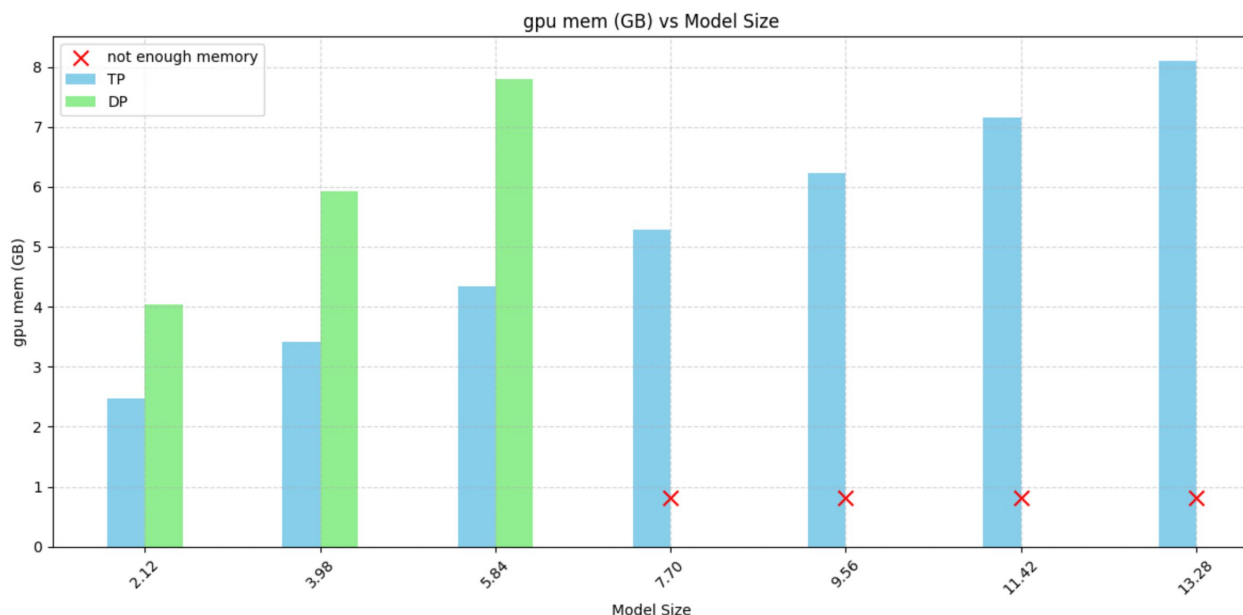


Рисунок 10. Сравнение тензорного параллелизма и параллелизма по данным по количеству потребляемой памяти на GPU для плотных моделей

Анализ графика показывает, что при относительно небольших моделях объём памяти, занимаемый на каждом GPU при использовании тензорного параллелизма, составляет примерно на 1.5 раза меньше, чем в случае параллелизма по данным. При увеличении размера модели разница в потреблении памяти становится ещё более

значительной и достигает почти двукратного снижения при использовании тензорного параллелизма. Это позволяет эффективнее использовать доступные вычислительные ресурсы и обучать значительно более крупные модели.

Однако, из рисунка 11 можно сделать вывод, что при использовании тензорного параллелизма на небольших моделях задержки выше, чем при использовании параллелизма по данным. Это связано с необходимостью выполнения коллективных операций, создающие задержки при инференсе.

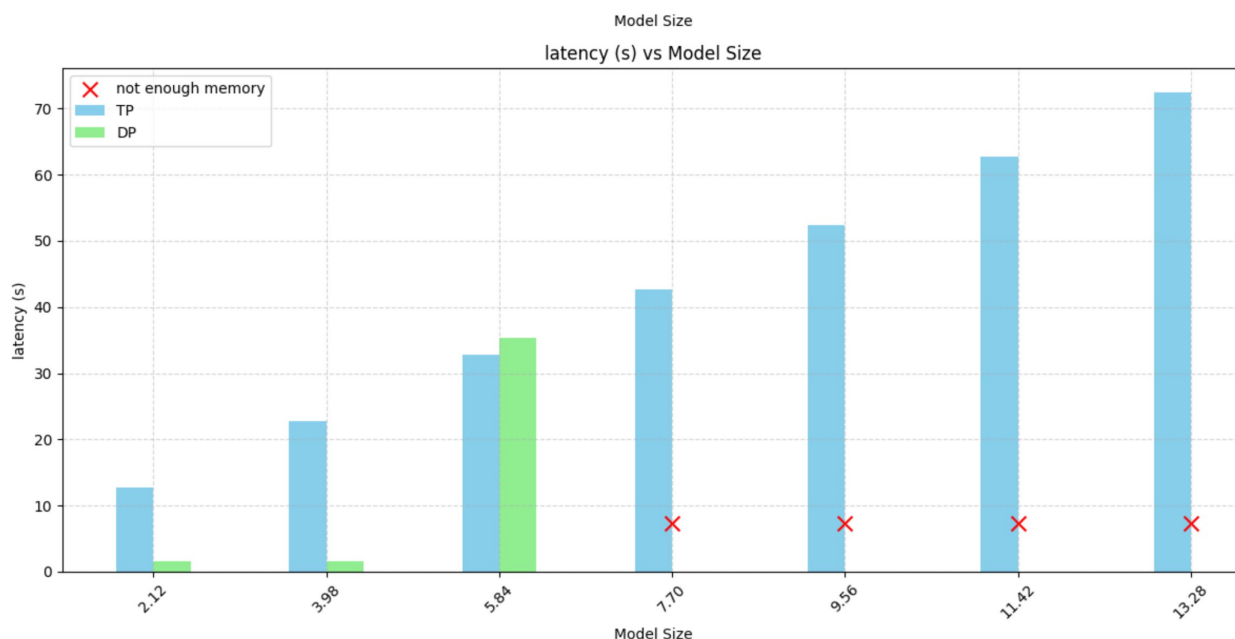


Рисунок 11. Сравнение тензорного параллелизма и параллелизма по данным по объему задержек для плотных моделей

Из этого можно сделать вывод, что тензорный параллелизм эффективно использовать на больших моделях, на более маленьких лучше параллелизм по данным.

4.2. Использование ZeRO с CPU и NVMe

Рассмотрим использование ZeRO при выгрузке параметров модели на диск и в RAM. На рисунке 12 представлено сравнение задержек инференса при выгрузке параметров на CPU и NVMe. Анализ графика демонстрирует, что задержки при использовании NVMe значительно превышают аналогичные значения при выгрузке в оперативную память CPU. Это обусловлено тем, что при работе с моделями небольшой величины отсутствует эффективное перекрытие операций межпроцессорной коммуникации вычислениями, что в свою очередь приводит к увеличению задержек

Таким образом, применение стратегии выгрузки параметров модели на NVMe оправдано лишь в случае работы с очень крупными моделями, где объем хранимых данных превышает объем доступной оперативной памяти, и использование высокоскоростного хранилища становится необходимым для обеспечения возможности масштабирования.

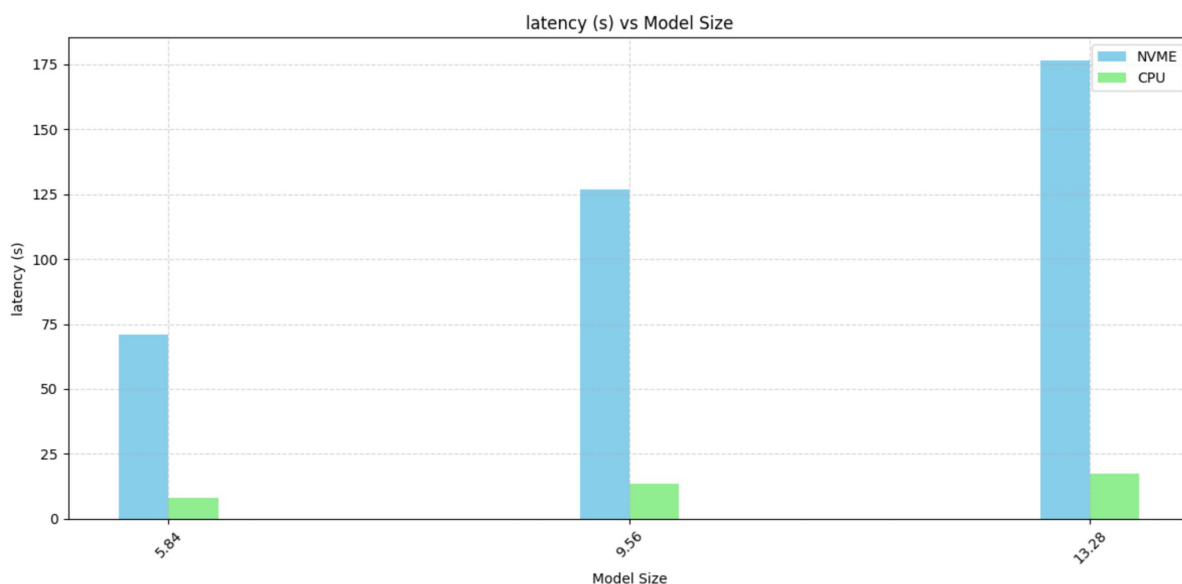


Рисунок 12. Сравнение задержек при выгрузке параметров модели на диск и в RAM

4.3. Использование тензорного параллелизма с ZeRO и без него

Рассмотрим применение тензорного параллелизма в сочетании с технологией ZeRO. На рисунке 13 приведено сравнение объёма видеопамяти, требуемого для каждой GPU при различных размерах модели. Из графика можно сделать вывод, что использование ZeRO приводит к значительному снижению объёма видеопамяти, необходимого на каждую отдельную GPU. При этом разность увеличивается при увеличении размера модели. К тому же, как видно из графика, ZeRO позволяет значительно увеличить размеров модели для инференса, при этом разница в потреблении памяти между сценарием с использованием ZeRO и без него возрастает по мере увеличения размера модели. Кроме того, как видно из представленных данных, применение ZeRO позволяет обрабатывать модели значительно большего размера в ходе инференса.

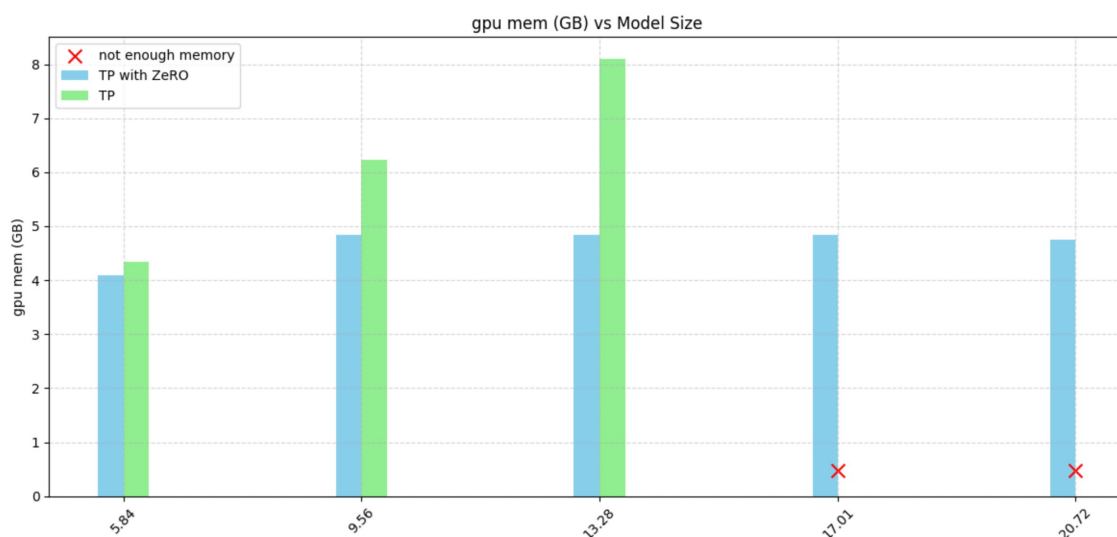


Рисунок 13. Сравнение тензорного параллелизма с ZeRO и без него по количеству потребляемой памяти на GPU для плотных моделей

К тому же, при анализе рисунка 14 можно сделать вывод, что благодаря технологии ZeRO могут быть уменьшены задержки. Следовательно, использование ZeRO в пределах одного узла с тензорным параллелизмом может привести к суперлинейному ускорению.

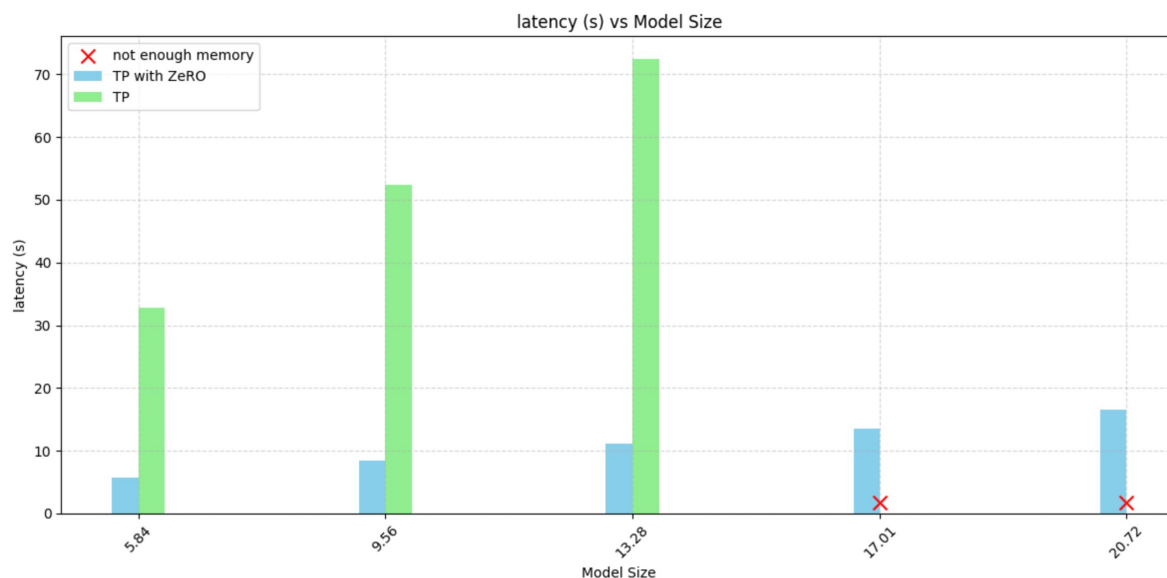


Рисунок 14. Сравнение задержек при использовании тензорного параллелизма с выгрузкой параметров и без

4.4. Использование параллелизма по данным и тензорного параллелизма с ZeRO.

Рассмотрим применение технологии ZeRO в комбинации с тензорным параллелизмом и параллелизмом по данным. На рисунке 15 представлено сравнение задержек, возникающих при выполнении инференса с использованием выгрузки параметров модели, для указанных стратегий параллелизма.

Анализ графика позволяет сделать вывод о том, что сочетание тензорного параллелизма с технологией ZeRO обеспечивает меньшие задержки на этапе инференса по сравнению с использованием параллелизма по данным в аналогичных условиях.

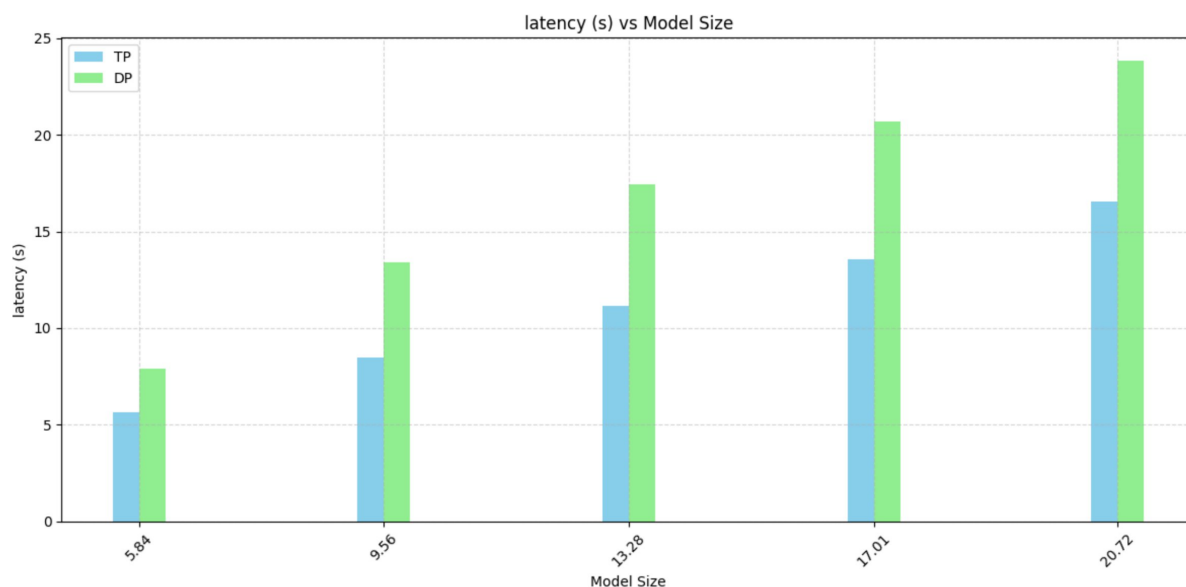


Рисунок 15. Сравнение объема задержек при использовании ZeRO с тензорным параллелизмом и параллелизмом по данным

4.5. Использование тензорного параллелизма вместе с экспертным параллелизмом.

Использование Megatron-LM вместе с DeepSpeed-MoE [16] позволяет комбинировать экспертный параллелизм с экспертным. На рисунке 16 представлено сравнение двух подходов — чистого экспертного параллелизма и комбинации тензорного параллелизма с MoE-архитектурой — по объёму видеопамети, потребляемой на одну GPU. Анализ графика показывает, что применение тензорного параллелизма позволяет сократить объём используемой памяти примерно в 1.4 раза.

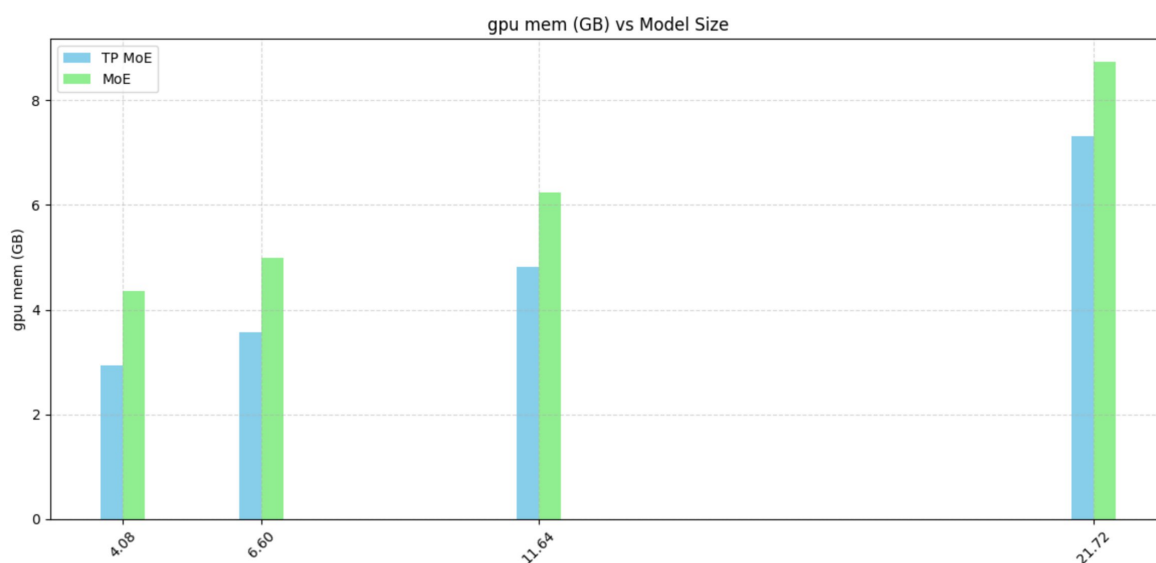


Рисунок 16. Сравнение использование экспертного параллелизма с тензорным и без по количеству потребляемой памяти

Кроме того, согласно данным, представленным на рисунке 17, задержки при выполнении инференса в целом одинаковы.

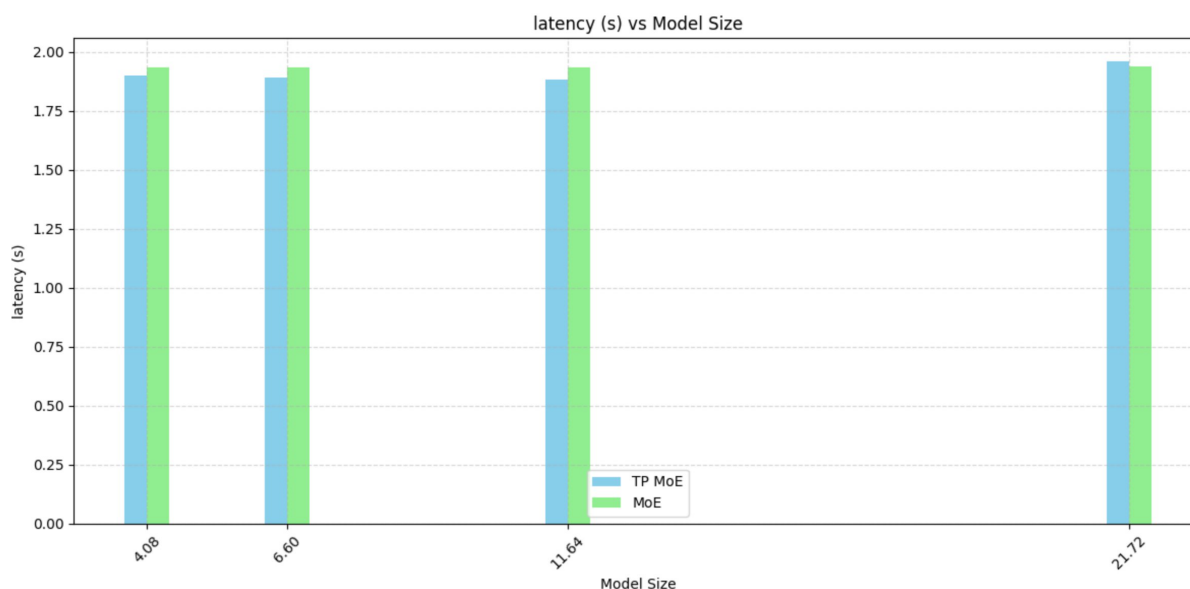


Рисунок 17. Сравнение использования экспертного параллелизма с тензорным и без по объёму задержек

Из этого можно сделать вывод, что комбинация этих видов параллелизма является эффективным решением с точки зрения количества потребляемой памяти.

Вывод

В ходе выполнения данной работы исследованы различные способы параллелизма моделей и изучены их основные преимущества и недостатки. Рассмотрены подходы к оптимизации распределённого инференса, включая выгрузку параметров модели из памяти GPU, оптимизацию коллективных коммуникационных операций, а также перераспределение токенов и экспертов между вычислительными ускорителями.

Проведён анализ существующих фреймворков, предоставляющих инструменты для реализации распределённого инференса. Для каждого из рассмотренных выявлены ключевые достоинства и недостатки.

Проведены практические эксперименты с тензорным параллелизмом, параллелизмом по данным и экспертным параллелизмом с использованием оптимизации ZeRO и без. И показано, что:

1. Тензорный параллелизм позволяет значительно снизить объём видеопамати, необходимой на каждом GPU, что делает возможным размещение более крупных моделей по сравнению с параллелизмом по данным. При этом при достаточной сложности модели увеличение задержек оказывается незначительным или отсутствует вовсе.
2. Из-за плохого перекрытия коммуникаций между RAM и дисковой памятью на небольших моделях использование ZeRO с выгрузкой параметров на диск может оказаться значительно менее эффективным в сравнении с выгрузкой только в RAM.
3. Применение оптимизации ZeRO в сочетании с тензорным параллелизмом не только даёт возможность использовать более большие модели, но и уменьшает задержки при инференсе.
4. Сочетание тензорного параллелизма и оптимизации ZeRO демонстрирует меньшие задержки по сравнению с параллелизмом по данным.
5. Использование тензорного параллелизма вместе с экспертным позволяет сократить объём потребляемой памяти на каждом GPU, тем самым даёт возможность использовать более большие модели. При этом объёмы задержек примерно такие же как и с использованием только MoE параллелизма.

Список использованной литературы

1. Distributed Parallel Native // mindspore URL: https://www.mindspore.cn/docs/en/r2.3.1/design/distributed_training_design.html (дата обращения: 15.11.2024).
2. Глубокое обучение // getsomemath URL: http://getsomemath.ru/subtopic/deep_learning/mlp/intro_to_mlp (дата обращения: 15.11.2024).
3. GPU Performance Background User's Guide // nvidia URL: <https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html> (дата обращения: 10.12.2024)
4. Mastering LLM Techniques: Inference Optimization // nvidia URL: <https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/> (дата обращения: 15.12.2024)
5. Andres Rodriguez. Deep Learning Systems. - 1-е изд. - San Rafael, California: Morgan And Claypool
6. Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism // arxiv.org. - 2020
7. DeepSpeed // DeepSpeed URL: <https://www.deepspeed.ai/> (дата обращения: 03.06.2025).
8. Megatron-LM // HuggingFace URL: https://huggingface.co/docs/accelerate/usage_guides/megatron_lm (дата обращения: 03.06.2025).
9. ZeRO inference // DeepSpeed URL: <https://www.deepspeed.ai/2022/09/09/zero-inference.html> (дата обращения: 03.06.2025).
10. Megatron-LM GPT2 // DeepSpeed URL: <https://www.deepspeed.ai/tutorials/megatron/> (дата обращения: 03.06.2025).
11. Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, Yuxiong He DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale // arxiv.org. - 2022
12. Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, Yuxiong He ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning // arxiv.org. - 2021
13. DeepNVMe // Github URL: <https://github.com/deepspeedai/DeepSpeed/blob/master/blogs/deepspeed-gds> (дата обращения: 03.06.2025).
14. Mixture of Experts Explained // HuggingFace URL: <https://huggingface.co/blog/moe> (дата обращения: 03.06.2025).
15. Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, Zhifeng Chen GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding // 2020
16. Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He DeepSpeed-MoE:

Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale // 2022

17. Yan Li, Pengfei Zheng, Shuang Chen, Zewei Xu, Yuanhao Lai, Yunfei Du, Zhengang Wang Speculative MoE: Communication Efficient Parallel MoE Inference with Speculative Token and Expert Pre-scheduling // 2025

Приложение А

Для изменения размера плотной модели варьировалось количество слоёв. В архитектуре МоЕ, помимо этого, также изменялось число экспертов. Все остальные параметры оставались неизменными:

- Размерность вектора скрытого состояния (H): 4096
- Число голов: 32
- Длина последовательности токенов : 1024
- Максимальный размер последовательности: 2048
- Размер слоя FFN (FFN size, F): 4096
- Максимальный размер позиционного эмбединга
- Размер батча: 4
- Размер словаря (V): 50372
- Размер параметра (float 16): 2 байта
- Количество слоев (N)
- Число экспертов (E)

Формула для вычисления плотной трансформерной модели (GB):

$$Size = \frac{2 * (4 * (H^2 + H) + 2 * H * F + 3 * H) * N + V * H + H * M}{2^{30}} \approx 0.186 * N + 0.26$$

Формула для вычисления модели с архитектурой МоЕ:

$$Size = \frac{2 * (4 * (H^2 + H) + E * (2 * H * F + 4 * H)) * N + V * H + H * M}{2^{30}} \approx 0.063 * N * E + 0.13 * N + 0.26$$

Приложение В

Таблица 1. Параллелизм по данным

Размер модели	Число слоев	Количество потребляемой памяти на GPU (GB)	Задержки (сек)	Пропускная способность (токен/сек)
2.12	10	4.045	1.595	15.049
3.98	20	5.921	2.975	8.066
5.84	30	7.796	35.308	1.813

Таблица 2. Тензорный параллелизм

Размер модели	Число слоев	Количество потребляемой памяти на GPU (GB)	Задержки (сек)	Пропускная способность (токен/сек)
2.12	10	2.469	12.757	5.017
3.98	20	3.407	22.805	2.806
5.84	30	4.345	32.713	1.956

7.70	40	5.283	42.612	1.502
9.56	50	6.221	52.456	1.220
11.42	60	7.159	62.790	1.019
13.28	70	8.097	72.455	0.883

Таблица 3. ZeRO

Пространство для выгрузки	Размер модели	Число слоев	Количество потребляемой памяти на GPU (GB)	Задержки (сек)	Пропускная способность (токен/сек)
NVMe	5.84	30	2.186	71.044	0.338
NVMe	9.56	50	2.187	126.881	0.189
NVMe	13.28	70	2.189	176.881	0.136
CPU	5.84	30	6.999	7.900	3.038
CPU	9.56	50	8.688	13.391	1.792
CPU	13.28	70	8.689	17.440	1.376
CPU	17.01	90	8.659	20.690	1.160
CPU	20.72	110	8.660	23.828	1.007

Таблица 4. Тензорный параллелизм с ZeRO

Размер модели	Число слоев	Количество потребляемой памяти на GPU (GB)	Задержки (сек)	Пропускная способность (токен/сек)
5.84	30	4.095	5.657	4.243
9.56	50	4.846	8.480	2.830
13.28	70	4.847	11.169	2.149
17.01	90	4.848	13.579	1.767
20.72	110	4.755	16.559	1.449

Таблица 5. Параллелизм по данным с ZeRO

Размер модели	Число слоев	Количество потребляемой памяти на GPU (GB)	Задержки (сек)	Пропускная способность (токен/сек)
5.84	30	6.999	7.900	3.038
9.56	50	8.688	13.391	1.792
13.28	70	8.689	17.440	1.376
17.01	90	8.659	20.690	1.160
20.72	110	8.660	23.828	1.007

Таблица 6. Экспертный параллелизм

Размер модели	Число слоев	Число экспертов	Количество потребляемой памяти на GPU (GB)	Задержки (сек)	Пропускная способность (токен/сек)
4.08	10	4	4.358	1.934	12.408
6.60	10	8	4.983	1.935	12.403
11.64	10	16	6.234	1.934	12.408
21.72	10	32	8.735	1.937	12.391
7.90	20	4	6.546	3.049	6.563
12.94	20	8	7.797	3.681	6.519

Таблица 7. Экспертный и тензорный параллелизм

Размер модели	Число слоев	Число экспертов	Количество потребляемой памяти на GPU (GB)	Задержки (сек)	Пропускная способность (токен/сек)
4.08	10	4	2.938	1.899	12.641
6.60	10	8	3.563	1.892	12.684
11.64	10	16	4.814	1.882	12.752
21.72	10	32	7.315	1.959	12.252
7.90	20	4	4.345	3.450	6.957
12.94	20	8	5.595	3.412	7.033
22.02	20	16	8.097	3.415	7.027